HDOC DAY 72

PROBLEM

Develop a C program to manage student attendance using a dynamically resizable matrix. The program must use pointers for matrix handling and must not use a fixed-size 2D array. Each row of the matrix represents one student, and each column represents one working day. Use 1 to indicate "Present" and 0 to indicate "Absent". The program must:

- Accept the initial number of students and working days.
- Use malloc() to dynamically allocate memory for the initial attendance matrix.
- Accept the attendance details of the initial students.
- Allow additional students to be added dynamically.
- Use realloc() to increase the matrix size when new students are added.
- Accept only 0 or 1 as attendance input.
- Calculate and display the attendance percentage of each student.
- Use separate functions for:
    - Creating the initial matrix
    - Recording attendance
    - Adding students
    - Displaying attendance records
    - Calculating attendance percentages
    - Releasing allocated memory
- Handle invalid input and memory-allocation failure appropriately.
- Release all dynamically allocated memory using free() before program termination.

CODE

```
#include <stdio.h>
```

```
#include <stdlib.h>

// Function declarations
int *createMatrix(int students, int days);
void recordAttendance(int *attendance, int start, int students, int days);
int *addStudents(int *attendance, int *students, int days, int newStudents);
void displayAttendance(int *attendance, int students, int days);
void calculatePercentage(int *attendance, int students, int days);
void releaseMemory(int *attendance);

int main()
{
   int students, days, newStudents;
   int *attendance;

   // Initial number of students
   printf("Enter number of students: ");
   if (scanf("%d", &students) != 1 || students <= 0)
   {
      printf("Invalid number of students.\n");
      return 1;
   }

   // Number of working days
   printf("Enter number of working days: ");
   if (scanf("%d", &days) != 1 || days <= 0)
   {
      printf("Invalid number of working days.\n");
```

```
    return 1;
}

// Create initial attendance matrix
attendance = createMatrix(students, days);

if (attendance == NULL)
{
    printf("Memory allocation failed.\n");
    return 1;
}

// Record initial attendance
recordAttendance(attendance, 0, students, days);

printf("\nInitial Attendance Record:\n");
displayAttendance(attendance, students, days);

// Add students
printf("\nEnter number of new students to add (0 for none): ");

if (scanf("%d", &newStudents) != 1 || newStudents < 0)
{
    printf("Invalid input.\n");
    releaseMemory(attendance);
    return 1;
}
```

```
if (newStudents > 0)
{
   int *temp;

   temp = addStudents(attendance, &students, days, newStudents);

   if (temp == NULL)
   {
      printf("Memory reallocation failed.\n");
      releaseMemory(attendance);
      return 1;
   }

   attendance = temp;
}

// Display final matrix
printf("\nFinal Attendance Record:\n");
displayAttendance(attendance, students, days);

// Calculate attendance percentages
printf("\nAttendance Percentages:\n");
calculatePercentage(attendance, students, days);

// Release dynamically allocated memory
releaseMemory(attendance);

return 0;
```

```
}


// Creates the initial attendance matrix
int *createMatrix(int students, int days)
{
   int *matrix;

   matrix = (int *)malloc(students * days * sizeof(int));

   return matrix;
}


// Records attendance
void recordAttendance(int *attendance, int start,
                int students, int days)
{
   int i, j, value;

   for (i = start; i < students; i++)
   {
      printf("\nEnter attendance for Student %d:\n", i + 1);

      for (j = 0; j < days; j++)
      {
         while (1)
         {
```

```
            printf("Day %d (1 = Present, 0 = Absent): ", j + 1);

            if (scanf("%d", &value) != 1)
            {
               printf("Invalid input. Enter only 0 or 1.\n");

               // Clear invalid input
               while (getchar() != '\n');
               continue;
            }

            if (value == 0 || value == 1)
            {
               *(attendance + i * days + j) = value;
               break;
            }

            printf("Invalid attendance. Enter only 0 or 1.\n");
         }
      }
   }
}


// Adds new students using realloc()
int *addStudents(int *attendance, int *students,
             int days, int newStudents)
{
```

```
    int oldStudents = *students;
    int totalStudents = oldStudents + newStudents;

    int *temp = (int *)realloc(
        attendance,
        totalStudents * days * sizeof(int)
    );

    if (temp == NULL)
    {
        return NULL;
    }

    *students = totalStudents;

    printf("\nEnter attendance for the new students:\n");

    recordAttendance(temp, oldStudents, totalStudents, days);

    return temp;
}


// Displays attendance matrix
void displayAttendance(int *attendance, int students, int days)
{
    int i, j;
```

```
    printf("\nStudent\t");

    for (j = 0; j < days; j++)
    {
        printf("D%d\t", j + 1);
    }

    printf("\n");

    for (i = 0; i < students; i++)
    {
        printf("%d\t", i + 1);

        for (j = 0; j < days; j++)
        {
            printf("%d\t", *(attendance + i * days + j));
        }

        printf("\n");
    }
}


// Calculates attendance percentage
void calculatePercentage(int *attendance, int students, int days)
{
    int i, j;
```

```
    for (i = 0; i < students; i++)
    {
        int present = 0;

        for (j = 0; j < days; j++)
        {
            if (*(attendance + i * days + j) == 1)
            {
                present++;
            }
        }

        double percentage = (present * 100.0) / days;

        printf("Student %d: %.2f%%\n", i + 1, percentage);
    }
}


// Releases dynamically allocated memory
void releaseMemory(int *attendance)
{
    free(attendance);
}
```